Confidential mode v2 vs v3: one two-party handshake per connection, not per request

v2 is retired; nodes and clients speak only v3 (`PROTOCOL-V3.md`). Numbers: loopback, Apple M2 Max, Venice.

The short version. Both versions hide prompts and answers from the node, keep the node's API key from the client, pin the model and token cap, and bill from the usage the provider reports. v2 paid a two-party computation (2PC) of the TLS keys for **every request**. v3 pays it once per **connection**. Over HTTP/2 the node writes its API key once, at the start; the provider keeps it in its header table (HPACK) and each later request, written by the client, names it with one byte. Without usable HTTP/2, v3 runs v2's flow, one request per connection, inside the same lifecycle.

	v2	**v3**
Unit	a session = one request	a connection that carries requests
HTTP to the provider	HTTP/1.1	framing `"2"` (HTTP/2, sequential requests) or `"1.1"` (one request), from the node's signed offer, in the operator's order
2PC	per request; 22.12 MB down, 1.21 MB up	per connection; 16.38 MB down, 1.13 MB up (22 garbled SHA-256 compressions instead of 31)
Client's proof before the provider may start	π_C2S: the last record is the suffix	π_WIN: HTTP/2 frame headers, the whole header record and the suffix; on HTTP/1.1 the suffix record
Held at the setup	a full-context reservation	a connection deposit (default at least 1 sat)
Reservation; session fee	per session; per request	per request; once per connection
Pre-warm	a session bound to model, token cap and body length	a connection bound to the credential only; on HTTP/2 a kept connection replaces it
Images	always the cold cost, about 26 MB	one more request on an open HTTP/2 connection: 36 kB up, image + 9 kB down
A cheating node learns at most	2.8 bits per request	3 bits per connection

Data and speed. The HTTP/2 rows were measured with the v2-size 2PC, before the smaller key schedule was merged; with it an opening is about 6 MB smaller (`[TO MEASURE]`). Not measured yet: a framing `"1.1"` request, and any of this over a remote link end to end (`[TO MEASURE]`).

	v2	v3
First chat request	about 28 MB	opening an HTTP/2 connection: 23.7 MB down, 2.6 MB up, 1.0–1.1 s
Each later chat request	about 28 MB again	0.5–1.0 MB down, about 85 kB up
Time to first token through routstrd	pre-warmed 0.9–1.5 s	2.0 s on the opening request, 0.46–1.26 s on a reused connection
Cold 2PC: local; 2 Mbit/s up, 50 down; 2 up, 10 down (simulated)	0.67 s; 7.42 s; 20.12 s	0.45 s; 7.02 s; 15.61 s
2PC online round trips; node peak memory	1 + 1; 517 MiB	3 + 1.5; 457 MiB

Money. At the setup the node holds a **connection deposit** instead of a full-context reservation. It is released with the first settled request, and charged when a connection is opened and wasted (no request settled, and the failure was not the node's or the provider's). Each request has its own reservation, sized as v2's, and its own receipt. The node closes an idle connection after `idle_timeout_s`.

Security. Unchanged: the trust model, the roles, the joint key share, the garbling construction, π_NK, π_C3S, the suffix, the attestation and the receipt. New or changed:

- *The credential sits in the provider's HPACK table*; the client names it by index. No HPACK encoding returns a stored value: tables are per direction, matches are on whole fields, push is off, and the client's header blocks are proven byte for byte.
- *The node holds every client record until π_WIN verifies* its frame header and last byte (so TLS padding cannot hide a frame), the whole header block and the suffix. A second request cannot be smuggled in.
- *The key schedule makes the inner HMAC states public* and adds label-check pairs; two more assumptions.
- *Linkability.* Requests on one HTTP/2 connection are linkable for the provider. The client's `reuseScope` separates what must not be linked; routstrd separates by local API key by default.
- *Not reviewed.* π_WIN, the frame rules and the key schedule: no external review. One provider tested.

Upgrading. v3 only: a v3 node answers a v2 client with `upgrade_required`, a v3 client reports `node_protocol_outdated` for a v2-only node. Neither falls back.

- **Operator:** update the node and the sidecar together. In the sidecar offer file list the framings in your order of preference: `"http": ["2", "1.1"]` with an `h2` block, or `["1.1"]`. Before listing `"2"`, run `provider-profile-check.py --h2` against your provider. Set the deposit and the idle timeout, or keep the defaults (`docs/SETUP.md`).
- **Client:** update routstrd (or the SDK) with `cu-2pc` and `cu-prover`; no configuration change. The client falls back from `"2"` to `"1.1"` on the same node when an HTTP/2 connection fails before a body was released. The local OpenAI API of routstrd is unchanged.